

CSE 354



Distributed Computing

Lecture 8

Presented by Dr. Hossam AbdelRahman

CHAPTER 08:

Fault Tolerance

Dependability

Basics:

- A component provides services to clients.
- To provide its service, a component might rely on services from other components.
- **Key idea:** A component can **depend** on other components.

Definition:

- A component **C** depends on **C*** if the **correctness of C's behavior** relies on the **correctness of C***.
- Components can be **processes** or **communication channels**.

Example:

- Consider a **banking system**:
 - A **transaction service** depends on the **account database service**.
 - If the database service fails or gives incorrect data, the transaction service may fail or behave incorrectly.

Dependability Requirements:

- Reliability
- Availability
- Safety
- Security
- Maintainability

Requirment	Description
Availability	Readiness for usage
Reliability	Continuity of service delivery
Safety	Very low probability of catastrophes
Maintainability	How easy can a failed system be repaired

Reliability versus availability

Reliability $R(t)$:

- The **probability** that a component **C** has been working correctly during the interval $[0, t)$, given that it was working at time 0.
- In simple terms: *“How likely is it that the component keeps working without failure?”*

Key Metrics:

1. Mean Time To Failure (MTTF):

- Average time until a component fails.
- **Example:** A hard drive has an MTTF of 100,000 hours → on average, it works 100,000 hours before failing.

2. Mean Time To Repair (MTTR):

- Average time to fix a failed component.
- **Example:** If a server crashes, MTTR might be 2 hours to restore it.

3. Mean Time Between Failures (MTBF):

- Total expected time between one failure and the next.
- **Formula:** $MTBF = MTTF + MTTR$
- **Example:** If a machine works 1000 hours (MTTF) and takes 10 hours to repair (MTTR), $MTBF = 1010$ hours.

Availability:

- How often the component is **ready for use**, considering both reliability and repair time.
- **High reliability doesn't guarantee high availability** if repair takes too long.

Working period → Failure → Repair → Working period

Reliability versus availability

Availability of a Component

Definition:

- **Availability** $A(t)$ of component **C**:
 - The **average fraction of time** that C is **up and running** during the interval $[0, t)$.
- **Long-term availability**: $A = A(\infty)$

Formula:

$$A = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}} = \frac{\text{Mean Time To Failure}}{\text{Mean Time Between Failures}}$$

Example:

- A web server has:
 - MTTF = 1000 hours
 - MTTR = 10 hours
- Availability $A = \frac{1000}{1000+10} \approx 0.99 \rightarrow$ server is **up 99% of the time**

Important Observation:

- Reliability and availability metrics **only make sense if we clearly define what a failure is**.
 - **Example:** Is a slow response a failure, or only a crash?
 - Defining failures consistently is key for accurate metrics.

Reliability versus availability

Metric	Meaning	Example
MTTF	Mean Time To Failure – average working time before failure	Server works 1000 hrs on average before crashing
MTTR	Mean Time To Repair – average downtime to fix a failure	Takes 10 hrs to repair the server
MTBF	Mean Time Between Failures = MTTF + MTTR	$1000 + 10 = 1010$ hrs
Availability (A)	Fraction of time the component is up	$A = \text{MTTF} / \text{MTBF} = 1000/1010 \approx 0.99 \rightarrow 99\%$ uptime

Terminology

Key Terms:

Term	Description	Example
Failure	When a component does not meet its specifications	A program crashes unexpectedly
Error	The part of a component that can lead to a failure	A wrong calculation or corrupted data
Fault	The underlying cause of an error	A programming bug or hardware defect

Concept:

- Fault tolerance is the ability of a system to **continue operating correctly even in the presence of faults**.
- Understanding the chain:
Fault → Error → Failure

Example Scenario:

- A programmer writes code with a bug (**fault**).
- During execution, the bug causes an incorrect calculation (**error**).
- Eventually, the program crashes (**failure**).

Terminology – Handling Faults

Term	Description	Example
Fault Prevention	Prevent faults from occurring in the first place	Hire competent programmers, follow coding standards
Fault Tolerance	Design components to continue working even when faults occur	Have each module implemented independently by two programmers → system can mask a bug
Fault Removal	Reduce the presence, number, or severity of faults	Conduct code reviews and testing, replace underperforming staff
Fault Forecasting	Estimate current faults, future occurrence, and impact	Evaluate how likely a recruiter is to hire sloppy programmers and predict potential software issues

Terminology – Types of Failures

Type of Failure	Description	Example
Crash Failure	Component halts unexpectedly, but works correctly until it stops	A web server runs fine, then suddenly crashes
Omission Failure	Fails to send or receive messages or respond	- Receive omission: Server doesn't receive a request - Send omission: Server fails to send a response - Response omission: Server fails to respond to a client request
Timing Failure	Response occurs too early or too late, outside the expected time window	A payment gateway confirms a transaction after the timeout period
Response Failure	Component responds, but the response is incorrect	Server returns wrong calculation for a client request
Value Failure	The value of the response is wrong	A database query returns incorrect data
State-Transition Failure	The system deviates from its expected flow of control	A vending machine dispenses two items for one coin
Arbitrary (Byzantine) Failure	Component may produce any output at any time, including malicious behavior	A compromised node sends contradictory messages to different clients

Halting Failures

Scenario:

- Component **C** stops perceiving any activity from component **C***.
- Is this a **halting failure**?
- Distinguishing between a **crash** or an **omission/timing failure** may be impossible.

System Models:

- **Asynchronous System:**
 - No assumptions about process execution speeds or message delivery times
 - Cannot reliably detect crash failures
- **Synchronous System:**
 - Process speeds and message delivery times are **bounded**
 - Can reliably detect omission and timing failures
- **Partially Synchronous System:**
 - Most of the time behaves like a synchronous system
 - There is no bound during asynchronous periods
 - Can usually detect crash failures reliably

Illustration:

C* activity → C perceives it

|

∨

Halting? Crash? Timing?

Asynchronous: cannot detect crash

Synchronous: can detect crash

Partially synchronous: usually can detect crash

Terminology – Types of Halting Failures

Type	Description
Fail-stop	Crash failures that are reliably detectable
Fail-noisy	Crash failures that are eventually reliably detectable
Fail-silent	Omission or crash failures where clients cannot tell what went wrong
Fail-safe	Arbitrary failures that are benign and cannot cause harm
Fail-arbitrary	Arbitrary failures that can be malicious

Redundancy for Failure Masking

Types of Redundancy:

- **Information Redundancy:**
 - Add extra bits to data so that errors can be detected and corrected.
 - Example: Error-correcting codes in memory or network packets.
- **Time Redundancy:**
 - Design the system so an action can be **performed again** if it fails.
 - Typically used for **transient or intermittent faults**.
 - Example: Retransmitting a network packet if it gets lost.
- **Physical Redundancy:**
 - Add **extra equipment or processes** so that one or more components can fail without affecting the system.
 - Widely used in distributed systems.
 - Example: Multiple servers in a cluster for high availability.

Illustration:

Information → extra bits

Time → retry operations

Physical → extra servers/components

Process Resilience

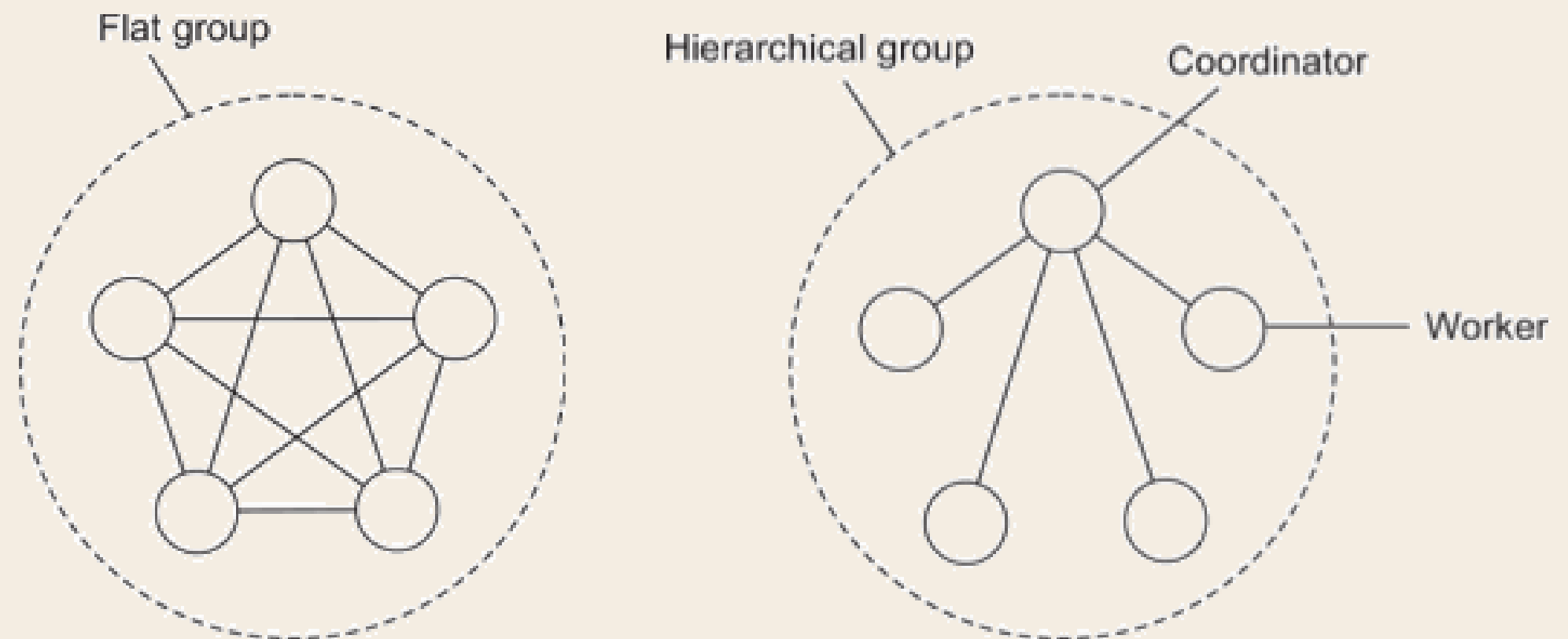
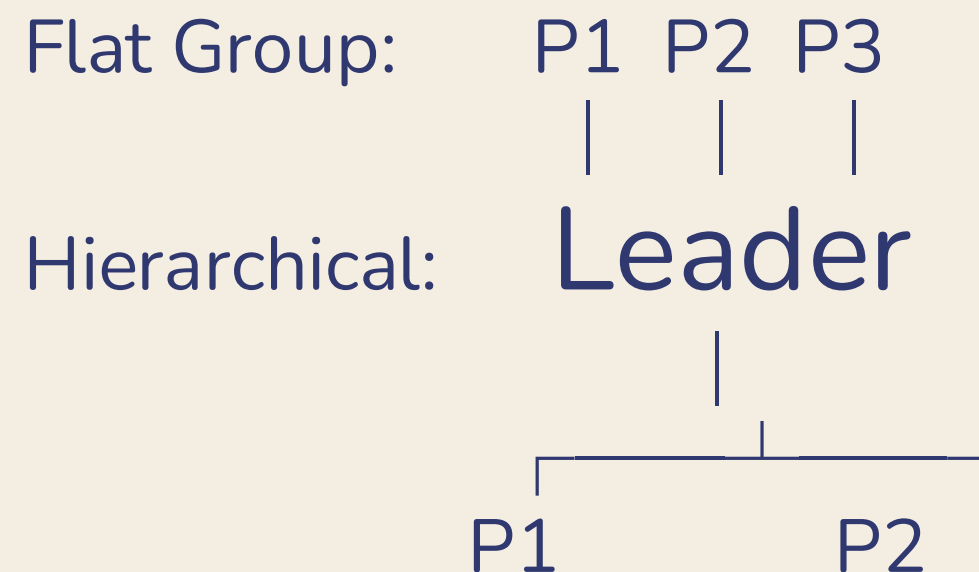
Basic Idea:

- Protect against malfunctioning processes using **process replication**.
- Organize multiple processes into a **process group**.

Types of Process Groups:

- **Flat Groups:** All processes are at the same level and provide the same service.
- **Hierarchical Groups:** Processes are organized in levels, often with leaders coordinating subgroups.

Illustration:



Groups and failure masking

k-Fault Tolerant Group:

- A group is **k-fault tolerant** if it can mask **any k concurrent member failures**.
- **k** = degree of fault tolerance

Group Size Requirements:

- **Halting failures (crash/omission/timing):**
 - Need **$k + 1$ members**
 - Reason: No member produces incorrect results, so a single correct result is enough
- **Arbitrary failures (Byzantine):**
 - Need **$2k + 1$ members**
 - Reason: Correct result is obtained through a **majority vote**

Important Assumptions:

- All members are **identical**
- All members **process commands in the same order**

Result:

- All processes produce **exactly the same outputs**, ensuring consistency despite failures

Illustration:

Halting: $k=1 \rightarrow 2$ members $\rightarrow 1$ can fail

Arbitrary: $k=1 \rightarrow 3$ members $\rightarrow$ majority vote

All members identical & process in same order $\rightarrow$ same output

Consensus in Fault-Tolerant Systems

Prerequisite:

- In a fault-tolerant process group, each **nonfaulty process executes the same commands in the same order** as every other nonfaulty process.

Reformulation:

- Nonfaulty group members must **reach consensus on which command to execute next**.

Illustration:

Process Group:



Consensus ensures all nonfaulty processes stay synchronized.

Consensus under Arbitrary Failure Semantics

Essence:

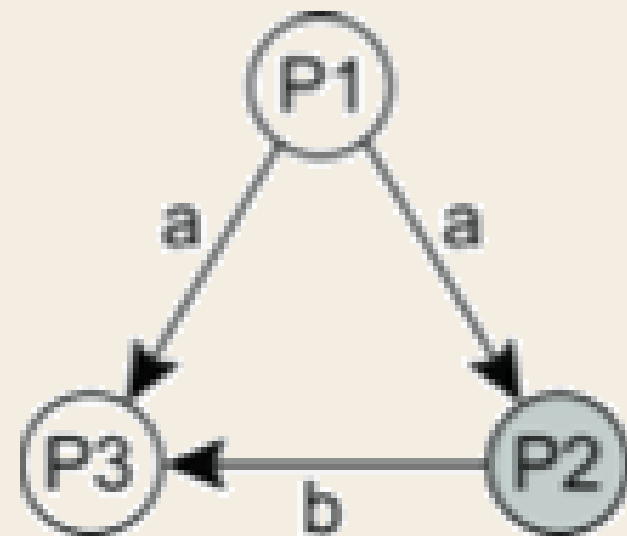
- Process groups are considered where **communication between processes may be inconsistent**.
- Processes may receive **different or conflicting messages** due to arbitrary (Byzantine) failures.
- The goal remains for **nonfaulty processes to reach agreement** despite these inconsistencies.

Illustration:

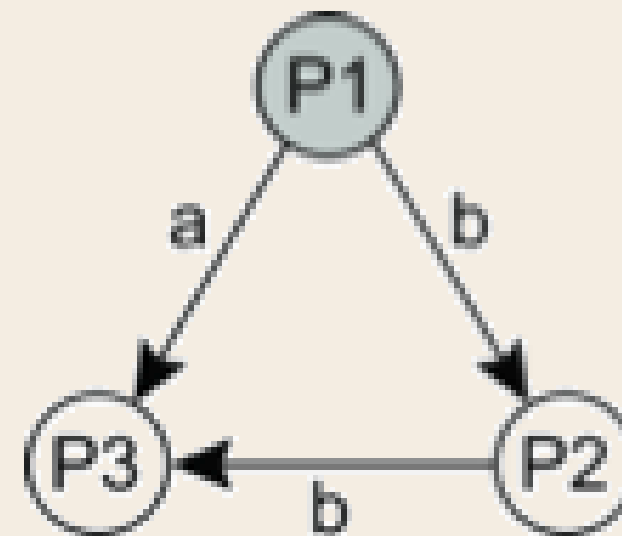
P1 → P2: command A

P1 → P3: command B (message altered)

Nonfaulty processes must agree on a single command despite discrepancies.



Improper forwarding



Different messages

Consensus under arbitrary failure semantics

System Model:

- A **primary process (P)** and **$n - 1$ backup processes ($B_1, \dots, B_{n-1}$)**.
- A client sends a value $v \in \{T, F\}$ to the primary.
- **Message assumptions:**
 - Messages may be **lost**, but losses can be detected.
 - Messages cannot be **corrupted beyond detection**.
 - Receivers can reliably **identify the sender**.

Byzantine Agreement Requirements:

- **BA1:** Every **nonfaulty backup** stores the **same value**.
- **BA2:** If the **primary is nonfaulty**, every nonfaulty backup stores **exactly what the primary sent**.

Observation:

- **Faulty primary:** BA1 ensures backups agree among themselves, but they may store a **wrong value**.
- **Nonfaulty primary:** Satisfying BA2 automatically ensures BA1 is satisfied.

Illustration:

Client $\rightarrow$ P: v

P $\rightarrow$ $B_1, B_2, \dots, B_{n-1}$

Primary faulty? $\rightarrow$ backups may agree on wrong value (BA1)

Primary correct? $\rightarrow$ backups store correct value (BA2 $\Rightarrow$ BA1)

Why $3k$ Processes Are Not Enough

Why $3k$ Processes Are Not Enough for Byzantine Failures

- Consider k **faulty processes** in a system.
- To tolerate **arbitrary (Byzantine) failures**, $2k + 1$ **correct processes** are needed for a majority vote.
- Total processes required: $3k + 1$, not just $3k$.

Reason:

- With only $3k$ processes, k **faulty processes** can prevent the correct majority:
 - Example: $k = 1 \rightarrow 3k = 3$ processes
 - 1 faulty, 2 correct $\rightarrow$ faulty can confuse one correct process, majority cannot be guaranteed
- With $3k + 1$ **processes**, at most k are faulty, leaving $2k + 1$ **nonfaulty** $\rightarrow$ ensures a **majority of correct processes** for agreement.

Illustration:

Total = $3k$

Faulty = k

Nonfaulty = $2k$

$\rightarrow k$ faulty can split votes $\rightarrow$ consensus fails

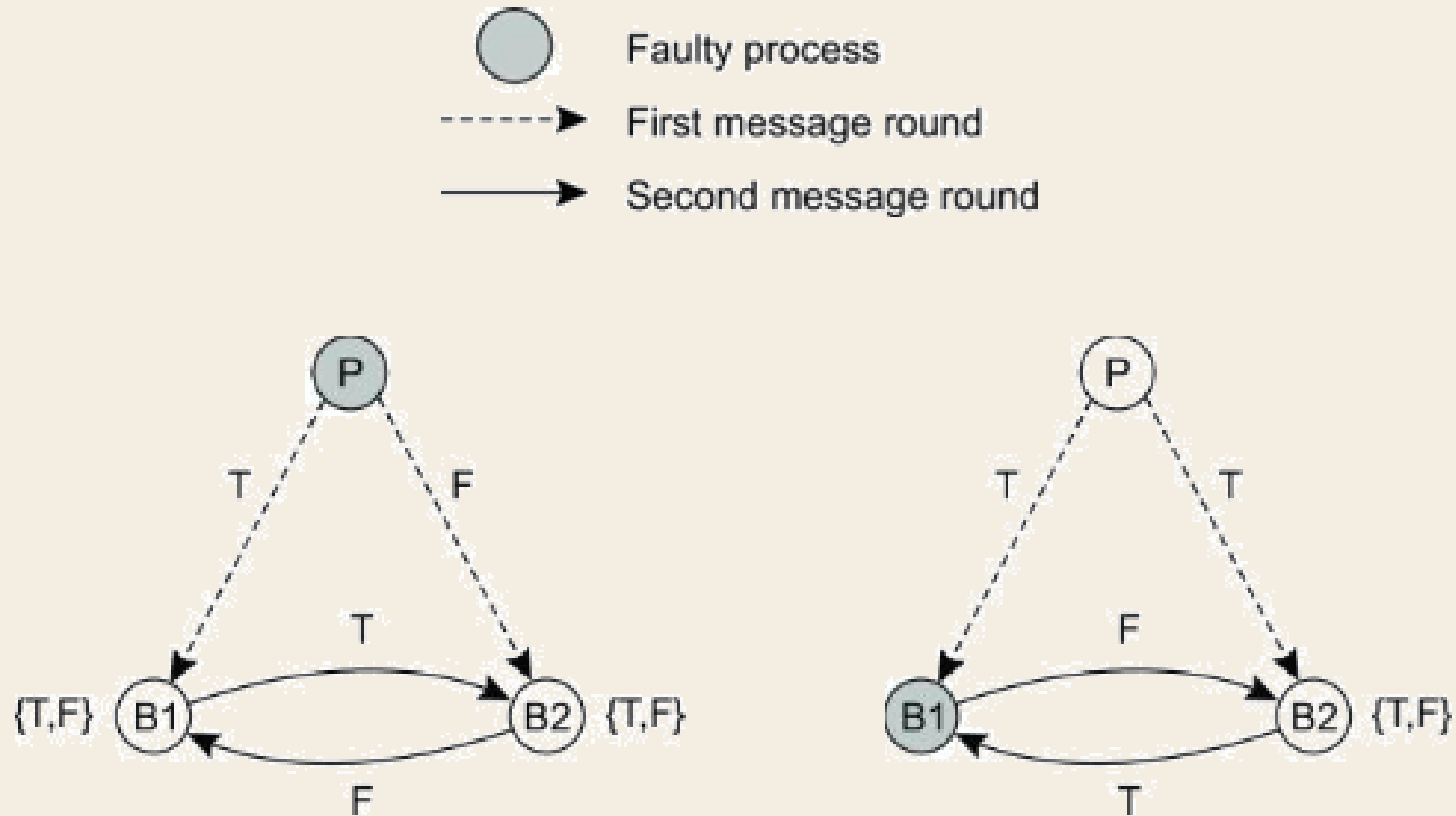
Total = $3k+1$

Faulty = k

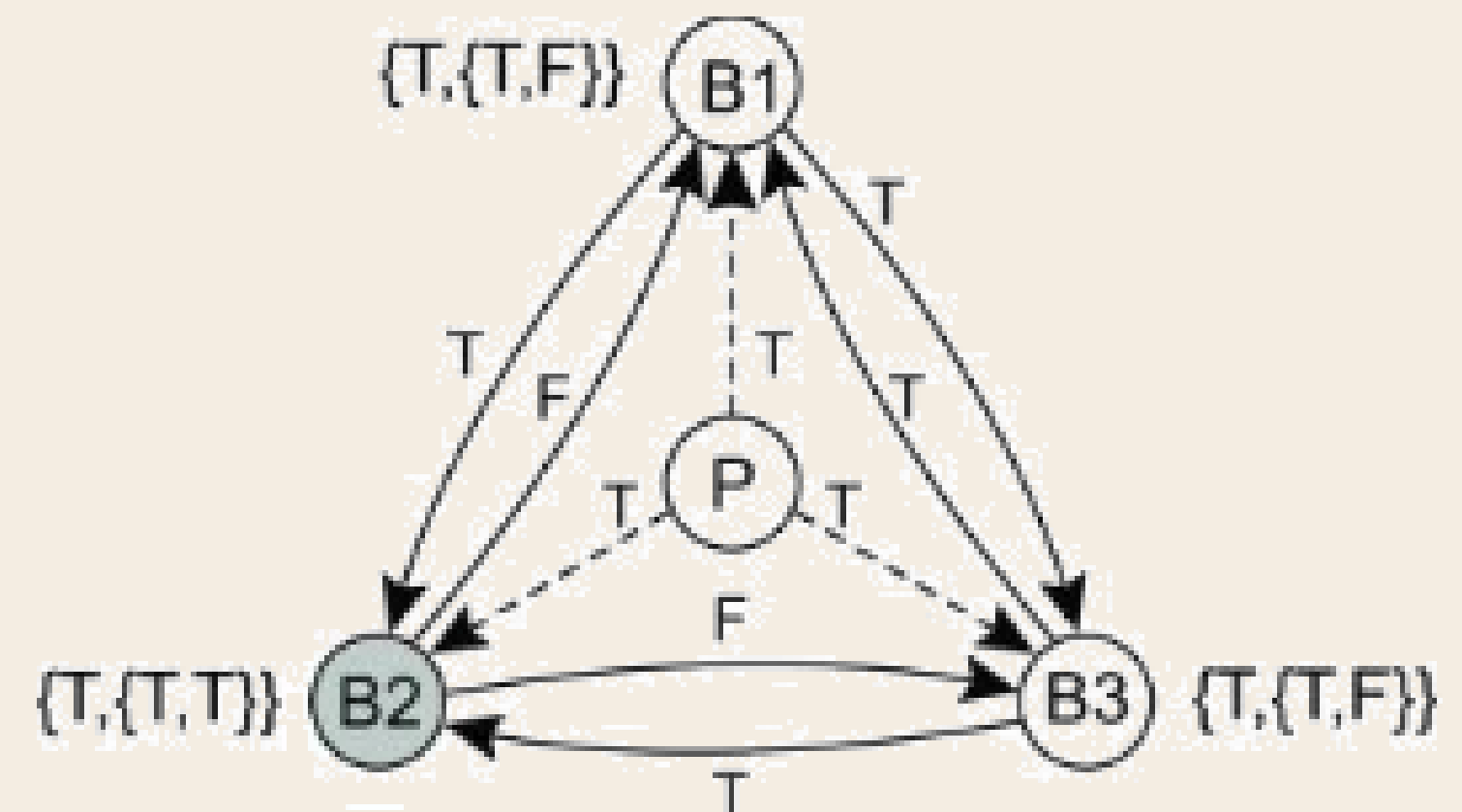
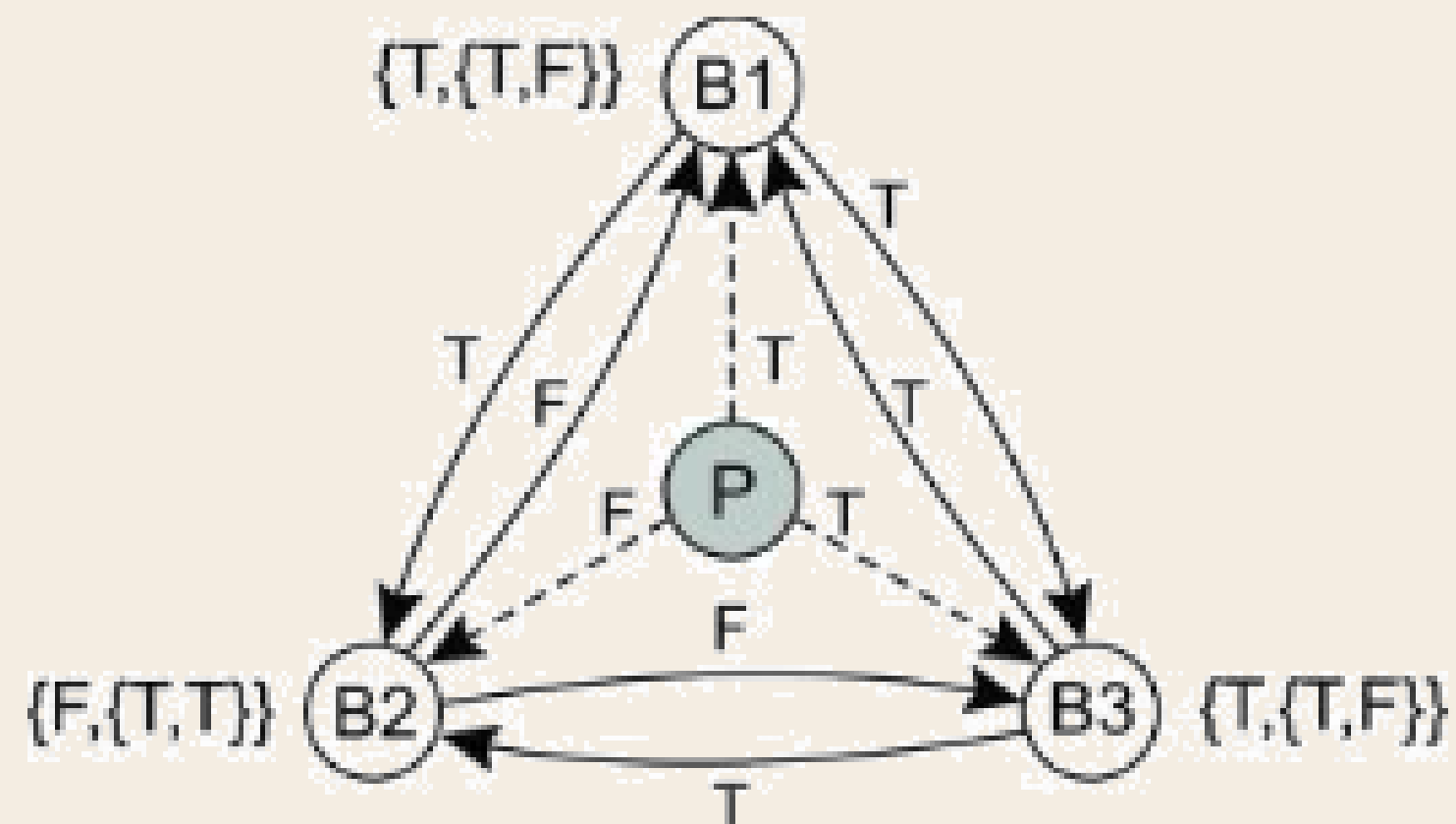
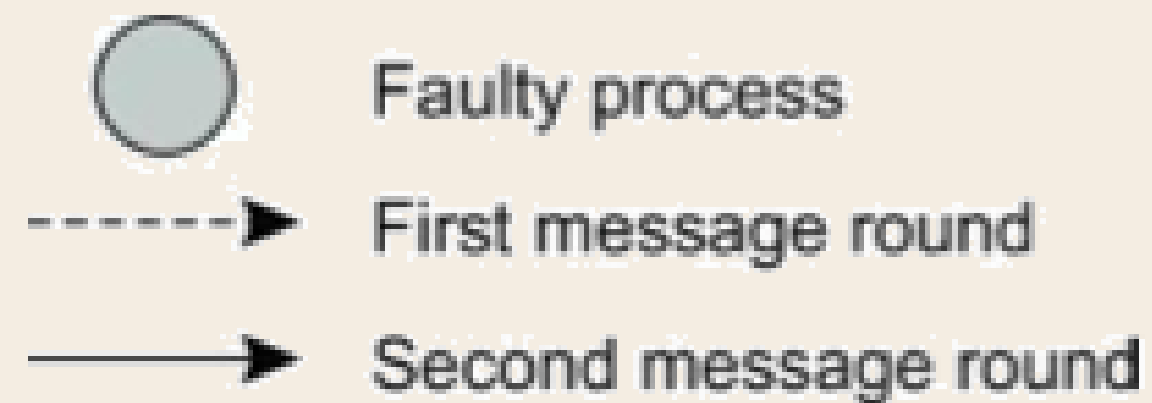
Nonfaulty = $2k+1$

$\rightarrow$ correct majority $\rightarrow$ consensus succeeds

Why 3k Processes Are Not Enough



Why $3k + 1$ Processes Are Enough



Consistency, availability, and partitioning

CAP Theorem:

Any networked system providing **shared data** can provide **at most two** of the following three properties:

Consistency (C):

All copies of a shared data item appear as a **single, up-to-date copy**

Availability (A):

Every request eventually **receives a response**

Partition Tolerance (P):

The system continues to operate correctly even if the **network is partitioned** (communication failures occur)

Conclusion:

In a network subject to communication failures, it is **impossible to guarantee** an atomic read/write shared memory that responds to every request while maintaining full consistency.

Illustration:

Choose any 2 of 3:

C + A → can't tolerate partitions

C + P → may be unavailable

A + P → may be inconsistent

Failure detection

Issue:

- How can we **reliably detect** that a process has actually crashed?

General Model:

- Each process has a **failure detection module**.
- **Probing mechanism:**
 - Process **P** probes another process **Q** for a response.
 - If **Q responds** → considered **alive** by P.
 - If **Q does not respond within t time units** → suspected to have **crashed**.

Observation (Synchronous Systems):

- In a synchronous system, a **suspected crash** is equivalent to a **known crash** because time bounds are guaranteed.

Illustration:

P → probe → Q

↳ responds? → alive

↳ no response within t → suspected crash

Practical Failure Detection

Implementation:

Heartbeat mechanism:

Process **P** expects a heartbeat from process **Q** every **t time units**.

If **P does not receive a heartbeat** within **t** → **P suspects Q** has crashed.

Recovery:

If **Q later sends a message** and **P** receives it:

P stops suspecting Q

P increases the timeout value t to reduce false suspicions

Crash scenario:

If **Q truly crashed**, **P** will continue to **suspect Q** indefinitely.

Illustration:

Time →

Q sends heartbeat → received by P → alive

Missed heartbeat → P suspects Q

Message arrives later → suspicion cleared, t increased

Reliable remote procedure calls

Potential Failures:

1. Client cannot locate the server.
2. Request message from client to server is lost.
3. Server crashes **after receiving the request**.
4. Reply message from server to client is lost.
5. Client crashes **after sending the request**.

Two “Easy” Solutions:

1. **Cannot locate server:** Report back to the client.
2. **Request lost:** Simply **resend the message**.

Illustration:

Client → Request → Server

1. Not found → notify client
2. Request lost → resend
3. Server crashes → ?
4. Reply lost → ?
5. Client crashes → ?

Reliable RPC: server crash

Problem:

- **Normal case (a):** Request and reply succeed.
- **Crash scenarios (b) and (c):** Server may crash **before or after processing the request**.
- **Challenge:** The client cannot tell **what actually happened**.

Two Approaches:

1. At-least-once semantics:

- Server guarantees the operation is carried out **at least once**, even if it crashes.
- Risk: The operation may be **executed multiple times**.

2. At-most-once semantics:

- Server guarantees the operation is carried out **at most once**.
- Risk: The operation may **not be executed** if the crash occurs at a critical time.

Illustration:

Client → Request → Server

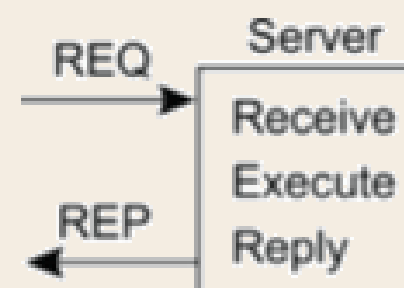
(a) Normal → processed & reply sent

(b) Crash before processing → may need retry

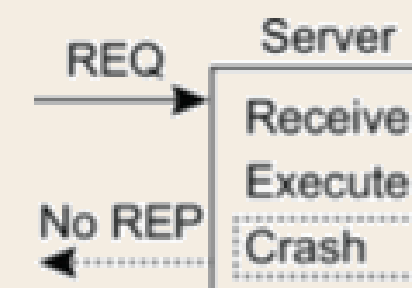
(c) Crash after processing → may cause duplicate execution

At-least-once → retry ensures execution

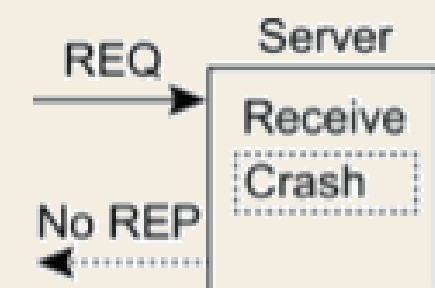
At-most-once → retry prevents duplicates



(a)



(b)



(c)

Reliable RPC: lost reply messages

The Issue:

- The client notices **no reply**, but cannot tell if it is due to:
 - A **lost request**
 - A **crashed server**
 - A **lost reply**

Partial Solution:

- Design the server so that operations are **idempotent**:
 - Repeating the operation has **the same effect** as executing it once.
 - Examples of idempotent operations:
 - **Pure read operations**
 - **Strict overwrite operations**
- Non-idempotent operations (e.g., many **banking transactions**) require **additional mechanisms** to avoid duplicates or inconsistencies.

Illustration:

Client → Request → Server (crash/lost reply)

Client retries →

Idempotent? → safe to repeat

Non-idempotent? → may cause duplicate effects

Reliable RPC: Client Crash

Problem:

- The server may be **performing work and holding resources** for a client that has crashed.
- This is called an **orphan computation**.

Solutions:

1. Client-initiated cleanup:

- When the client recovers, it **kills or rolls back** its orphan computations.

2. Epoch-based cleanup:

- Client broadcasts a **new epoch number** upon recovery.
- Servers **kill any computations** associated with the previous epoch.

3. Time-bound computations:

- Require operations to complete within **T time units**.
- Computations exceeding T are automatically **removed**.

Illustration:

Client crashes → Server computes (orphan)

Client recovers → broadcasts new epoch

Server kills orphans from old epoch

Time-bound → server cleans up long-running operations

Simple reliable group communication

Intuition:

- A message sent to a **process group G** should be **delivered to every member** of G.

Key Distinction:

- **Receiving a message:** Message arrives at a process (e.g., placed in a buffer).
- **Delivering a message:** Message is **accepted and processed** by the application.

Why this matters:

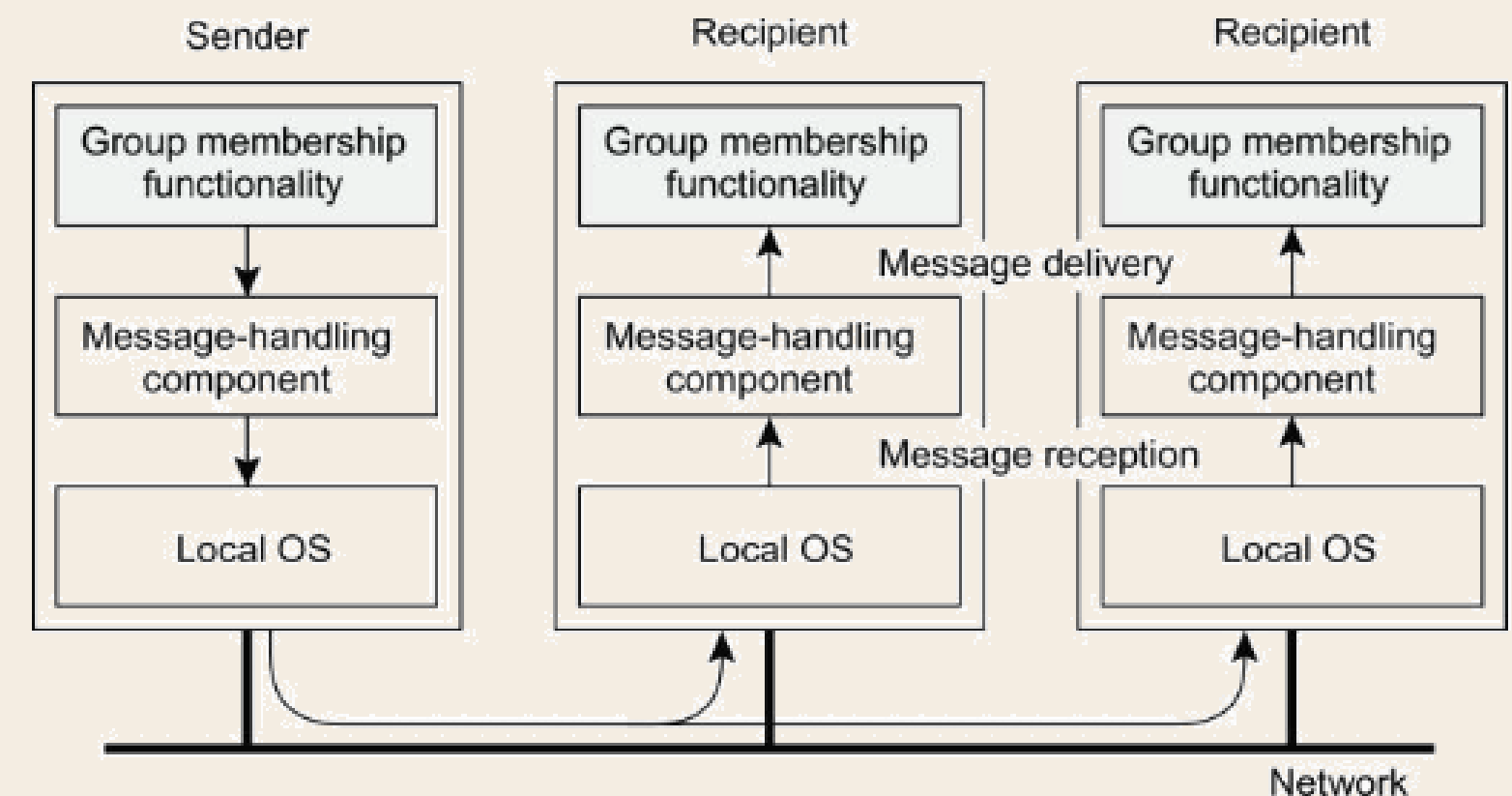
- A process may **receive** a message but not **deliver** it (e.g., due to errors or ordering constraints).
- Reliability focuses on ensuring **correct delivery**, not just reception.

Illustration:

Sender → Group G: {P1, P2, P3}

Receive: message arrives at each process

Deliver: message is accepted and processed by each process



Recovery: Background

Essence:

- When a failure occurs, the system must be brought back to an **error-free state**.

Types of Recovery:

- **Forward Error Recovery:**

- Move the system to a **new correct state** and continue execution.

- **Backward Error Recovery:**

- Roll back the system to a **previous error-free state**.

Practice:

- Most systems use **backward recovery**.
- Requires creating and maintaining **recovery points (checkpoints)**.

Observation:

- In distributed systems, recovery is more complex because:
 - Multiple processes must **coordinate**
 - They must agree on a **consistent global state** to roll back to

Illustration:

Failure occurs

↓

Forward recovery → move to new correct state

Backward recovery → rollback to checkpoint

Distributed challenge:

P1 }
P2 } → must agree on consistent recovery point
P3 }

Message logging

Alternative to Checkpointing:

- Instead of taking frequent (and expensive) checkpoints, we can **log messages**.
- After a failure, the system can **replay communication events** from the last checkpoint to recover.

Assumption: Piecewise Deterministic Execution Model

- Execution is divided into **state intervals**.
- Each interval starts with a **nondeterministic event** (e.g., receiving a message).
- Within each interval, execution is **deterministic**.

Conclusion:

- By **recording nondeterministic events** (such as message receipts), we can:
 - Replay them later
 - Reproduce the same execution
 - Achieve **deterministic recovery**

Illustration:

Checkpoint → Event1 → Event2 → Event3 → Failure

↓ log events

Recovery:

Checkpoint → replay Event1, Event2, Event3 → restored state

THANK YOU